

https://json-schema.org/ (Documentación oficial)

https://json-schema.org/learn/json-schema-examples (Ejemplos)

1 Introducción a JSON Schema 2
2 Estructura Básica de un JSON Schema 3
2.1 $schema 3
2.2 $id 3
2.3 Título (*title*) y descripción (*description*) 4
2.4 type 4
2.5 properties. 5
2.6 required. 6
2.7 additionalProperties 6
3 Validaciones para arrays 7
3.1 Esquemas de elementos 7
3.2 Propiedades adicionales utilizadas para arrays 8
4 Combinadores Lógicos 8
5 Referencias y reutilización 10
5.1 Referencias internas 10
5.2 Referencias externas 11
6 Esquemas condicionales if, then, else 13
6.1 Comparaciones Numéricas 13
6.2 Comparaciones con Valores Específicos 14
6.3 Presencia de Propiedades 14
6.4 Operaciones Lógicas 15
6.5 Condiciones basadas en estructura 16

1 Introducción a JSON Schema

JSON Schema es un **estándar** basado en JSON para **describir** y **validar** la estructura de documentos JSON. Su objetivo principal es proporcionar una forma clara y automática de validar este tipo de documentos, lo que ayudará a detectar errores y prevenir inconsistencias. Entre sus propósitos destacan los siguientes:

- **Validación de datos:** Garantiza que los datos recibidos cumplen con un formato específico y no contienen errores estructurales.
- **Documentación:** Describe de manera estructurada cómo debe ser el formato de datos, lo que facilita la comprensión por parte de desarrolladores y usuarios.
- **Interoperabilidad:** Ayuda a garantizar que diferentes sistemas o aplicaciones puedan intercambiar datos de manera coherente.
- **Generación de datos:** Permite generar formularios, interfaces de usuario o ejemplos de datos basados en el esquema.

El formato de un documento de JSON Schema se define en el propio formato JSON, por lo que resulta fácil de aprender y utilizar para quienes ya están familiarizados con este formato. **Permite dividir los esquemas en módulos** y reutilizarlos en diferentes proyectos, lo que fomenta la eficiencia. Además, es compatible con diversas herramientas de desarrollo, bibliotecas y lenguajes de programación.

Para resumir qué es JSON Schema podríamos decir que: **JSON Schema es a los documentos JSON, lo mismo que XML Schema a los documentos XML**. Un ejemplo de JSON Schema es el siguiente:

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "Persona",
  "description": "Un esquema para describir una persona.",
  "type": "object",
  "properties": {
    "nombre": {
      "type": "string",
      "description": "El nombre de la persona."
    },
    "edad": {
      "type": "integer",
      "description": "La edad de la persona en años.",
      "minimum": 0
    }
  },
  "required": ["nombre"]
}
```

Para validar un documento JSON respecto a un JSON Schema, podemos hacer uso de los múltiples validadores que existen, tanto en modo aplicación, como en modo librería. También, podríamos usar validadores online como: https://www.jsonschemavalidator.net/

2 Estructura Básica de un JSON Schema

Un JSON Schema describe de manera **jerárquica** la estructura de un documento JSON y utiliza palabras clave predefinidas para definir tipos, propiedades, y restricciones. A continuación, se describen los elementos que forman parte de esta tecnología.

2.1 $schema

Se utiliza para indicar la **versión de JSON Schema** utilizada. Es opcional, pero se recomienda incluirla para asegurar compatibilidad. Históricamente, se han utilizado una serie de valores:

- **Draft-04. ("http://json-schema.org/draft-04/schema#").** Una de las versiones iniciales que introdujo operadores lógicos como (*allOf*, *anyOf*, *oneOf*, *not*).
- **Draft-06. ("http://json-schema.org/draft-06/schema#").** Añadió soporte para formatos como *date-time*, *uri* y *hostname*. También, introdujo el uso de *$id*.
- **Draft-07. ("http://json-schema.org/draft-07/schema#").** Entre otras mejoras, añadió soporte para palabras clave como *if*, *then* y *else*.
- **Draft-2019-09. ("http://json-schema.org/2019-09/schema#").** Introdujo mejoras en modularización y reutilización.
- **Draft-2020-12. ("http://json-schema.org/2020-12/schema#").** Es la **última versión estable** y es compatible con la mayoría de herramientas modernas y **la recomendada** para proyectos nuevos.

Para especificar el valor de *$schema* en nuestro documento, lo haremos de la siguiente forma:

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema"
}
```

2.2 $id

Proporciona un **identificador único** para el esquema, lo cual es útil cuando se reutiliza o referencia desde otros esquemas. El valor de *$id* debe ser una **URI válida** que lo identifique, no siendo obligatorio que sea una URL accesible en la web.

Suele ser habitual, asignarle un valor que referencia al dominio de nuestra organización o proyecto, seguido de una estructura interna de organización. Por ejemplo:

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema"
  "$id": "https://iesAugustobriga.com/schemas/persona"
}
```

2.3 Título (*title*) y descripción (*description*)

Son metadatos opcionales que se utilizan para describir el esquema, asignándole un **título** y **descripción**.

```
{
  "title": "Persona",
  "description": "Describe información básica de una persona."
}
```

(OJO) En JSON, incluyendo JSON Schema, **no están permitidos los comentarios nativos** según la especificación estándar. Sin embargo, podemos añadir información contextual haciendo uso de la propiedad "**description**" o con propiedades añadidas por el propio usuario para este propósito y que es común que reciba el nombre de **"_comment"**, aunque podría ser cualquier otro.

2.4 type

Define el **tipo principal** del dato que se valida. Puede tomar uno de los siguientes valores:

- **object**. Representa un objeto JSON que contiene pares clave-valor, que se definirán posteriormente dentro de **"properties"**.

```
{
  "type": "object",
  "properties": {
    "nombre": { "type": "string" },
    "edad": { "type": "integer" }
  },
  "required": ["nombre"]
}
```

- **array**. Representa una lista de elementos. Para especificar el tipo de los elementos, usaremos **"items"**.

```
{
  "type": "array",
  "items": {"type": "string"}
}
```

- **string**. Representa una cadena de texto. Es posible definir características de ese texto con propiedades como:
  - **"minLength"**. Define el número mínimo de caracteres que debe tener.
  - **"maxLengt**h". Define el número máximo de caracteres que debe tener.
  - **"pattern"**. Define la expresión regular (ECMA-262) que la cadena debe cumplir.
  - **"enum"**. Define un conjunto cerrado de valores permitidos.
  - **"format"**. Valida que la cadena cumpla con formatos predefinidos como:

email date-time uri hostname ipv4 ipv6

```
{
  "type": "string",
  "enum": ["rojo", "verde", "azul"]
}
```

- **number**. Representa un número que incluye enteros y decimales.
- **integer**. Representa un número entero.

Existen una serie de propiedades que permiten restringir sus posibles valores:

  - **minimum**. Define el valor mínimo permitido
  - **exclusiveMinimum**. Define el valor mínimo permitido (no inclusivo).
  - **maximum**. Define el valor máximo permitido
  - **exclusiveMaximum.** Define el valor máximo permitido (no inclusivo)
  - **multipleOf.** Garantiza que el valor tenga que ser múltiplo del valor indicado. También se puede usar para restringir el número máximo de decimales que debe tener el número.

```
{
  "type": "number",
  "minimum": 0,
  "maximum": 10,
  "multipleOf": 0.01
}
```

- **boolean**. Representa un valor de *true* o *false*

- **null**. Representa un valor nulo.

2.5 properties.

Especifica las **propiedades** esperadas en un objeto JSON y sus características. Por ejemplo:

```
{
  "type": "object",
  "properties": {
    "nombre": { "type": "string" },
    "edad": { "type": "integer" }
  }
}
```

Debe quedar claro que, a diferencia de lo que ocurre en otros modelos, como las *secuencias* de *XML Schema*, el orden que establecemos en *"properties"* **no es relevante para la validación**.

Para el ejemplo anterior, serían válidos los dos documentos siguientes:

```
{
  "nombre": "Juan",
  "edad": 30
}
```

Y

```
{
  "edad": 30,
  "nombre": "Juan"
}
```

Si necesitáramos que las propiedades sigan un orden específico en el documento JSON, habría que manejarlo desde instancias externas, quedando fuera del alcance de JSON Schema.

2.6 required.

Permite establecer la lista de **propiedades obligatorias** en el objeto.

```
{
  "properties": {
    "nombre": { "type": "string" },
    "edad": { "type": "integer" }
  }
  "required": ["nombre"]
}
```

2.7 additionalProperties

La palabra clave **additionalProperties** en JSON Schema se utiliza para controlar si se permiten **propiedades adicionales** en un objeto que no estén definidas en la parte de **properties**. Admite varios valores:

- **additionalProperties: true (valor por defecto)**.

  Permite que el objeto tenga propiedades no especificadas en la definición de "**properties**" y no se validarán con ningún tipo de restricción.

- **additionalProperties: false**.

  Garantiza que no haya propiedades adicionales a las definidas en **"properties"**. Si el objeto contiene propiedades no listadas, el documento será inválido.

- **additionalProperties: {}.**

  Permite definir un **esquema** para validar las propiedades adicionales. Es útil si deseamos controlar las propiedades no listadas, pero con reglas específicas. Por ejemplo:

```
{
  "type": "object",
  "properties": {
    "nombre": { "type": "string" },
    "edad": { "type": "integer" }
  },
  "additionalProperties": {
    "type": "string"
  }
}
```

3 Validaciones para arrays

Los **arrays** en JSON Schema se definen mediante el valor ***"array"*** dentro de ***"type"***. A continuación, debemos especificar las propiedades que los elementos dentro de ese array deben cumplir, a través de la propiedad ***"items"***.

```
{
  "type": "array",
  "items": {
     "type": "string"
  }
}
```

3.1 Esquemas de elementos

El esquema básico con **un único tipo de elemento** dentro del array se define utilizando la propiedad **ítems** y especificando dentro de ella con ***"type"*** el tipo asociado a los elementos.

```
{
  "type": "array",
  "items": {
     "type": "integer"
  }
}
```

Si deseamos permitir **múltiples tipos** para los elementos de un array, podemos usar **items** como un **array de esquemas**. En este caso, cada elemento en el array se corresponde a una posición en el array original y debe respetarse el orden definido.

```
{
  "type": "array",
  "items": [
      { "type": "string" },
      { "type": "integer" }
  ]
}
```

El esquema anterior validaría un array en el que el **primer elemento** debe ser una cadena (string) y el **segundo elemento** debe ser un entero (integer). Como, por ejemplo:

["apple", 10]

Sin embargo, la siguiente combinación no sería válida:

["apple", "banana"]

Si nos interesara definir un **array** que contenga elementos que pueden ser de tipo *string* o *integer*, sin importar el orden de esos elementos, podríamos hacerlo utilizando un array de tipos válidos como valor de la propiedad "*type*".

```
{
  "type": "array",
  "items": {
      "type": ["string","integer"]
  }
}
```

Así, conseguiríamos que cualquiera de las dos combinaciones vistas antes, fuera válida.

Existe también la posibilidad de especificar un esquema para **todos los elementos adicionales** que sigan después de los primeros elementos, utilizando la propiedad ***"additionalItems"***. Si definimos el siguiente esquema:

```
{
  "type": "array",
  "items": [
      { "type": "string" },
      { "type": "integer" }
  ],
  "additionalItems": { "type": "boolean" }
}
```

Serían válidos valores asignados en los que el primer elemento fuera un string, el segundo un entero y el resto valores booleanos, como, por ejemplo:

["apple", 10, true, false]

3.2 Propiedades adicionales utilizadas para arrays

Es posible hacer uso de otras propiedades para ampliar las posibilidades a la hora de definir las características de nuestros arrays. Por ejemplo:

- ***minItems***. Especifica el número mínimo de elementos que puede tener el array.
- ***maxItems***. Especifica el número máximo de elementos que puede tener el array.
- ***uniqueItems***. Especifica si los elementos del array deben ser únicos (sin duplicados).
- ***contains***. Valida que al menos un elemento del array cumpla con un esquema específico.

4 Combinadores Lógicos

Estos se utilizan para combinar o restringir esquemas, permitiendo validar estructuras complejas con múltiples condiciones.

- **allOf**. Todos los esquemas especificados en una lista deben cumplirse.

```
{
  "type": "object",
  "allOf": [
    {
      "properties": {
        "nombre": { "type": "string" }
      },
      "required": ["nombre"]
    },
    {
      "properties": {
        "edad": { "type": "integer" }
      },
      "required": ["edad"]
    }
  ]
}
```

En este caso, el documento debería tener obligatoriamente una propiedad *"nombre"* y otra *"edad"*. El **orden de los esquemas dentro de allOf no es importante**. La palabra clave *allOf* simplemente indica que **todos los esquemas especificados deben cumplirse simultáneamente**, sin importar en qué orden aparezcan.

- ***anyOf.*** Al menos uno de los esquemas especificados debe cumplirse.
- ***oneOf***. Exactamente uno de los esquemas debe cumplirse.
- ***not***. El combinador **not** se usa cuando un documento **no debe cumplir** con el esquema especificado.

```
{
  "type": "object",
  "not": {
    {
      "properties": {
        "nombre": { "type": "string" }
      },
      "required": ["nombre"]
    },
  }
}
```

En el ejemplo anterior serían válidos documentos que no contengan la propiedad *"nombre"* o que la contengan con un tipo diferente al *"string"*. Recordemos que el valor de "**additionalProperties"** es *true* por defecto. Serían válidos:

```
{
"edad": 25
}
```

Y

```
{
"nombre": 25
}
```

Pero no:

```
{
"nombre": "Fernando"
}
```

5 Referencias y reutilización

El concepto de **referencias** y **reutilización** es uno de los aspectos más poderosos de *JSON Schema*, ya que permite estructurar esquemas de manera modular, manteniendo mayor limpieza en el código y evitando duplicaciones.

La palabra clave **$ref** se utiliza para incluir o "referenciar" un esquema externo o una parte de un esquema definido previamente.

```
{
  "$ref": "<URI o fragmento de esquema>"
}
```

Donde:

- **URI**: Puede apuntar a un archivo externo o URL.
- **Fragmento**: Referencia a una parte específica dentro del esquema actual usando un identificador **#**.

El uso de referencias aporta múltiples ventajas:

- Evita la duplicación de reglas en esquemas grandes.
- Permite dividir esquemas complejos en partes manejables y reutilizables.
- Consigue que los cambios en una definición se reflejan automáticamente en todos los lugares donde se usa.
- Garantiza que las validaciones sean uniformes en todo el esquema.
- Fomenta la colaboración al dividir esquemas en componentes que diferentes equipos pueden mantener.

5.1 Referencias internas

Podemos definir esquemas reutilizables dentro de una sección especial del esquema llamada **definitions** (o **$defs** en las versiones más recientes). Posteriormente, podremos referenciarlas desde otras partes de nuestro esquema con la sintaxis:

"$ref": "#/definitions/nombrePropiedad"

Por ejemplo:

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "definitions": {
    "Direccion": {
      "type": "object",
      "properties": {
        "calle": { "type": "string" },
        "ciudad": { "type": "string" },
        "codigoPostal": { "type": "string" }
      },
      "required": ["calle", "ciudad"]
    }
  },
  "type": "object",
  "properties": {
    "envio": { "$ref": "#/definitions/Direccion" },
    "facturacion": { "$ref": "#/definitions/Direccion" }
  }
}
```

5.2 Referencias externas

Podemos referenciar esquemas almacenados en archivos o URLs externas. Para ello, basta con referenciar explícitamente en el valor de *"$ref"*, la URL del contenido a referenciar. Por ejemplo, imaginemos que tenemos el siguiente fichero con el nombre ***"direccion.schema.json"***:

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "type": "object",
  "properties": {
    "calle": { "type": "string" },
    "ciudad": { "type": "string" },
    "codigoPostal": { "type": "string" }
  },
  "required": ["calle", "ciudad"]
}
```

Si ahora quisiéramos incluir el contenido del fichero anterior como parte de otro esquema, podríamos hacerlo de la siguiente forma:

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "type": "object",
  "properties": {
    "envio": { "$ref": "direccion.schema.json" },
    "facturacion": { "$ref": "direccion.schema.json" }
  }
}
```

También, es posible referenciar **una parte del contenido de un esquema externo** utilizando fragmentos de URI. Esto se logra con el símbolo **#,** que permite acceder a secciones específicas dentro de un esquema externo. Para ello, usaremos la siguiente sintaxis:

```
{
  "$ref": "<URL del esquema>#/ruta/a/la/parte"
}
```

No es obligatorio que el fichero externo referenciado use la propiedad *definitions* para poder referenciar parte de su contenido, pudiendo hacerlo sobre **cualquier parte del esquema**, siempre y cuando, tenga una ruta válida según las reglas de *JSON Pointer*. Sin embargo, usar *definitions* (o *$defs*) es una convención común que facilita la organización y reutilización de esquemas.

Por ejemplo, si tenemos el fichero ***direccion.schema.json*** con el siguiente contenido:

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "definitions": {
    "DireccionCompleta": {
      "type": "object",
      "properties": {
        "calle": { "type": "string" },
        "ciudad": { "type": "string" },
        "codigoPostal": { "type": "string" },
        "pais": { "type": "string" }
      },
      "required": ["calle", "ciudad", "codigoPostal"]
    },
    "CodigoPostal": {
      "type": "string",
      "pattern": "^[0-9]{5}$"
    }
  }
}
```

Podríamos referenciar parte de su contenido en otro documento de la siguiente forma:

```
{
 "$schema": "https://json-schema.org/draft/2020-12/schema",
 "type": "object",
 "properties":
 {
  "direccionEnvio":
  {
  "$ref": "direccion.schema.json#/definitions/DireccionCompleta"
  },
  "codigoPostalAlternativo":
  {
  "$ref": "direccion.schema.json#/definitions/CodigoPostal"
  }
 }
}
```

6 Esquemas condicionales if, then, else

Los **esquemas condicionales** en JSON Schema permiten definir **reglas que se aplican** dependiendo de **si ciertas condiciones son verdaderas o falsas**. Esto se logra con las palabras clave **if**, **then**, y **else.** Como ocurre en las diferentes tecnologías que utilizan esta estructura condicional, el uso del **else** es opcional.

Un ejemplo que utiliza esta estructura sería el siguiente:

```
{
  "type": "object",
  "properties": {
    "rol": { "type": "string" }
  },
  "if": {
    "properties": { "rol": { "const": "admin" } }
  },
  "then": {
    "required": ["adminID"],
    "properties": { "adminID": { "type": "string" } }
  },
  "else": {
    "if": {
      "properties": {"rol": { "const": "user" } }
    },
    "then": {
      "required": ["fechaNacimiento"],
      "properties": {
           "fechaNacimiento": {
                "type": "string",
                "format": "date"
           }
      }
    },
    "else": {
      "maxProperties": 1
    }
  }
}
```

Es posible combinar esquemas condicionales de forma anidada para manejar reglas más complejas.

6.1 Comparaciones Numéricas

Se usan palabras clave como **minimum**, **maximum**, **exclusiveMinimum** y **exclusiveMaximum** para establecer límites.

Ejemplo: Edad mínima de 18 años

```
{
  "if": {
    "properties": {
      "edad": { "minimum": 18 }
    }
  }
}
```

Ejemplo: Rango numérico (18 ≤ edad ≤ 65)

```
{
  "if": {
    "properties": {
      "edad": {
           "minimum": 18,
           "maximum": 65
      }
    }
  }
}
```

6.2 Comparaciones con Valores Específicos

Se usan **const** y **enum** para validar valores exactos o uno entre un conjunto de valores.

```
{
  "if": {
    "properties": {
      "rol": { "const": "admin" }
    }
  }
}
```

```
{
  "if": {
    "properties": {
      "estado": { "enum": ["activo", "inactivo"] }
    }
  }
}
```

6.3 Presencia de Propiedades

Se usa **required** para verificar si una propiedad está presente.

```
{
  "if": {
    "required": ["email"]
  }
}
```

6.4 Operaciones Lógicas

Las palabras clave **allOf**, **anyOf**, **oneOf** y **not** permiten construir condiciones lógicas más complejas.

- **allOf**: Condición "y". Todas las condiciones deben cumplirse.

```
{
  "if": {
    "allOf": [
      { "properties": { "edad": { "minimum": 18 } } },
      { "properties": { "edad": { "maximum": 65 } } }
    ]
  }
}
```

- **anyOf**: Condición "o". Al menos una de las condiciones debe cumplirse.

```
{
  "if": {
    "anyOf": [
      { "properties": { "tipo": { "const": "producto" } } },
      { "properties": { "tipo": { "const": "servicio" } } }
    ]
  }
}
```

- **oneOf**: Condición "xor". Sólo una de las condiciones debe cumplirse.

```
{
  "if": {
    "oneOf": [
      { "properties": { "estado": { "const": "activo" } } },
      { "properties": { "estado": { "const": "inactivo" } } }
    ]
  }
}
```

- **not**: La condición es verdadera si los datos no cumplen con el esquema.

```
{
  "if": {
    "not": {
       "properties": { "estado": { "const": "archivado" } }
    }
  }
}
```

6.5 Condiciones basadas en estructura

Las condiciones basadas en estructura permiten validar aspectos como:

- La cantidad de propiedades de un objeto.
- La cantidad de elementos en un array

Para chequear si un objeto tiene más de 3 propiedades podría poner:

```
{
  "type": "object",
  "if": {
    "minProperties": 4
  },
  "then": {
    …
  },
  "else": {
    …
  }
}
```

Para chequear si un array tiene más de 5 elementos podría poner:

```
{
  "type": "array",
  "if": {
    "minItems": 6
  },
  "then": {
    …
  },
  "else": {
    …
  }
}
```